

GeNeX: Genetic Network eXperts Framework for Addressing Validation Overfitting

Emmanuel Pintelas^{ID} and Ioannis E. Livieris^{ID}

Abstract—Excessive reliance on validation performance during model selection can lead to validation overfitting (VO), where models appear effective during development but fail at test time. This issue is further amplified in low-data regimes and under distribution shifts, where validation signals become unreliable. Although ensemble learning is widely used to improve robustness and generalization, most ensemble construction pipelines depend heavily on validation scores, leaving them vulnerable to VO and limiting their reliability in real-world deployment scenarios. To address this, we propose genetic network eXperts (GeNeXs), a framework that mitigates validation overfitting at both model generation and ensemble construction stages. In the generation phase, GeNeX employs a dual-path strategy: gradient-based training is coupled with genetic model evolution. Offspring networks are created via crossover of trained parents, promoting structural diversity and weight-level regeneration without relying on validation feedback. This results in a candidate pool of robust, nonoverfitted models. During ensemble construction, the candidate networks are clustered by prediction behavior to identify complementary model spaces. Within each cluster, multiple diverse experts are selected using criteria such as robustness and representativeness and fused at the weight level to form compact prototype networks. The final ensemble aggregates these prototypes, with predictions optimized via sequential quadratic programming (SQP) for output-level synergy. To rigorously evaluate VO resilience, we introduce a VO-aware evaluation protocol that simulates realistic deployment scenarios by enforcing distributional divergence between training and test subsets. Experiments on four real-world image classification tasks (Skin Cancer, DeepFake Detection, Plant Disease, and Pneumonia) demonstrate that GeNeX consistently outperforms state-of-the-art ensembles under limited-data and shift-aware conditions, achieving strong generalization with minimal VO gaps.

Index Terms—Deep learning, distribution shifts, ensemble learning, genetic algorithms (GAs), validation overfitting.

I. INTRODUCTION

VALIDATION overfitting (VO) consists of an underexplored, yet crucial challenge in deep learning model development [1], [2]. Unlike common training overfitting (TO), which occurs when a model memorizes the training data and fails to generalize, VO arises when a model appears optimal based on validation metrics but ultimately performs

poorly on truly unseen test data—particularly under distribution shifts where validation and test distributions diverge [2], [3]. This phenomenon is typically caused by excessive reliance on a limited or nonrepresentative validation set during model selection, hyperparameter tuning, or ensemble construction [4], leading to inflated validation scores that poorly reflect real-world performance.

The risk of validation overfitting increases significantly in scenarios characterized by repeated evaluations on the same validation set, particularly under limited-data conditions or in the presence of distribution shifts [2], [3]. In such cases, models may exploit statistical flukes in the validation set, leading to misleadingly high validation scores without true generalization capabilities [1], [5]. While validation monitoring is traditionally employed to avoid TO and select the best validation checkpoint, heavy dependence on validation feedback acts as a double-edged sword: it mitigates TO but simultaneously increases the risk of validation overfitting. Consequently, even models exhibiting small train-validation gaps and high validation accuracy may collapse when evaluated on new data [2].

This issue is particularly prevalent in real-world deployments with small or biased validation sets, research benchmarks in low-resource scenarios, and competition-driven environments [3]. Despite its significance, validation overfitting remains rarely addressed directly in deep learning pipelines, where model selection often continues to favor validation-optimized candidates. Such practices lead to the development of models that, despite strong validation performance, fail catastrophically when deployed [2].

Recent research has explored ensemble strategies aiming at improving generalization and robustness, including methods, which promote expert diversity [2], [3], employ genetic optimization search [6], [7], [8], or cluster models to identify experts based on predictive behavior [1], [5]. While these methods enhance diversity or structural complementarity, they often rely heavily on validation feedback during model selection. Moreover, most fail to explicitly control VO risk during model generation or ensemble construction. As a result, they remain vulnerable to VO, particularly in low-resource settings or under significant test-time distribution shifts—conditions commonly encountered in real-world deployments [2].

To address these limitations, we propose genetic network eXperts (GeNeXs), an ensemble framework designed explicitly to minimize the risk of validation overfitting while maintaining high generalization and robustness. GeNeX consists of two core modules: 1) a genetic-based ensemble (*GenE*) generator, which generates a diverse population of robust networks by combining supervised training and genetic weight fusion, with zero reliance on validation feedback;

Received 15 June 2025; revised 27 November 2025; accepted 26 February 2026. Date of publication 6 March 2026; date of current version 2 September 2026. (Corresponding author: Emmanuel Pintelas.)

Emmanuel Pintelas is with the Department of Statistics and Insurance Science, University of Piraeus, 18534 Piraeus, Greece (e-mail: e.pintelas@unipi.gr).

Ioannis E. Livieris is with the Department of Business and Organization Administration, University of Peloponnese, 24100 Kalamata, Greece (e-mail: livieris@uop.gr).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2026.3669867>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2026.3669867

and 2) prototype-based network eXperts (*ProtoNeX*) selector, which clusters the generated networks based on behavioral similarity, elects diverse experts within each cluster, and synthesizes prototype models through weight-level aggregation.

We hypothesize that incorporating a genetic-based weight-crossover mechanism into a standard gradient-based optimization pipeline can act as a “refreshing” regeneration process, mitigating overfitting in the networks’ weight level, while promoting diversity and robustness without validation dependency, thereby drastically reducing VO risk. Furthermore, constructing the ensemble via prototype-based expert fusion ensures complementary representation across the model space, enhancing the final ensemble’s stability and performance.

To rigorously validate this hypothesis, we designed a VO-aware evaluation protocol, which simulates realistic validation overfitting risks by deliberately enforcing distributional shift between training and testing sets. Rather than relying on random sampling, we introduce a Jensen–Shannon divergence (JSD)-guided partitioning algorithm that maximizes divergence between subsets while maintaining a predefined user data ratio (e.g., 30–70), enabling researchers to create VO-prone scenarios in any case-study dataset. JSD provides a symmetric and bounded measure of divergence, making it a robust choice for quantifying distributional differences [9]. In our protocol, JSD specifically acts as the optimization criterion during dataset splitting and subsequently serves as a direct indicator of VO susceptibility per task. High JSD implies strong distributional shifts between training/validation and testing distributions, indicating greater VO risk. Therefore, models that achieve high predictive performance while maintaining a low VO gap under high-JSD conditions demonstrate enhanced robustness and credibility in addressing VO.

Employing this controlled experimental setup, evaluation across four diverse, real-world image classification case studies consistently demonstrates GeNeX’s superior ability to achieve high predictive performance while effectively minimizing validation overfitting. Under challenging high-JSD scenarios, GeNeX significantly outperforms state-of-the-art ensemble baselines, underscoring its robustness and generalizability in mitigating VO.

The main contributions of this research work are summarized as follows.

- 1) We formally analyze the phenomenon of validation overfitting, quantifying its causes from both modeling and data perspectives.
- 2) We propose GeNeX, a two-phase ensemble learning framework that minimizes validation overfitting by combining genetic-based model generation with prototype-based clustering and fusion, achieving robust and high generalization performance under high VO risk.
- 3) We introduce a VO-aware evaluation protocol built on a dedicated JSD-guided data partitioning algorithm to simulate realistic distributional shifts for benchmarking VO resilience, offering significant utility for both academic research and industrial deployment pipelines.

II. DEFINITION OF VALIDATION OVERFITTING AND MOTIVATION

VO refers to the phenomenon where a model exhibits strong performance on a validation set during development but fails to

generalize to truly unseen test data [1]. Unlike classical TO, which results from excessive memorization of training data, VO arises when the model selection process becomes overly adapted to the specific characteristics, noise, or biases of the validation set.

The root cause of VO lies in the excessive reliance on validation feedback for model evaluation, selection, and early stopping. When a model is evaluated repeatedly against the same validation set, especially in low-data or imbalanced scenarios [10], the probability of observing artificially high validation scores by chance increases. As a result, models may be chosen not for their true generalization capabilities but for coincidental alignment with the validation data’s peculiarities.

Let $\mathcal{D}_T$, $\mathcal{D}_V$, and $\mathcal{D}_E$ denote the training, validation, and evaluation datasets, respectively, with $\mathcal{D}_V$ serving as a proxy to estimate generalization performance. In ideal conditions, performance on $\mathcal{D}_V$ should correlate strongly with performance on $\mathcal{D}_E$. However, under repeated evaluations and selection pressure, the following degradation can occur:

$$\text{Perf}(\mathcal{D}_T) \gg \text{Perf}(\mathcal{D}_V) \approx \text{Perf}(\mathcal{D}_E) \quad (\text{TO})$$

and

$$\text{Perf}(\mathcal{D}_V) \gg \text{Perf}(\mathcal{D}_E) \quad (\text{VO}).$$

In TO [11], the model performs well on training data but poorly on both validation and test data. In contrast, under validation overfitting, the model appears optimal on the validation set but fails dramatically on the test set, giving a false sense of reliability during development [2].

VO is particularly dangerous because it is often invisible until final testing or deployment phases, where operational data significantly diverges from validation samples. This risk is exacerbated when: the validation set is small, biased, or unrepresentative; the number of validation checks during training or model search is high; model selection strategies aggressively optimize for validation metrics. Thus, while validation monitoring remains a fundamental technique to avoid TO, over-dependence on validation evaluation without safeguards introduces a critical failure mode via VO.

In this context, minimizing the number of validation checks, promoting diversity in model generation, and decoupling model construction from heavy validation optimization emerge as promising strategies to mitigate validation overfitting risk. These insights directly motivate the design principles of the proposed GeNeX framework.

A. Formalizing Validation Overfitting

We formalize VO along two perspectives: 1) the *modeling perspective*, focusing on the pipeline of model generation and ensemble construction; and 2) the *dataset perspective*, where VO arises intrinsically due to underlying distributional shifts between development and evaluation data.

In the modeling view, VO emerges during both the generation of base models (often involving heavy validation monitoring) and the strategy used to select ensemble members. This includes widely used sequential selection methods that incrementally select ensemble members based on validation performance gains, a common approach in both practice and competitive settings [4]. On the dataset side, VO risk increases when the validation set is not representative of the

testing distribution, especially in real-world or competition-style environments with intrinsic domain or acquisition shifts [2], [3].

1) *Model-Induced VO Risk*: To promote diversity, a pool of N models is typically generated via N independent training runs, each performed under varying configurations (e.g., random seeds, data augmentations, and dropout schemes). Each model is trained using a standard train-validation loop, during which validation performance is monitored across q checkpoints (e.g., per epoch), and the best-performing checkpoint is retained. This process results in a total of $N \times q$ validation queries during the model generation phase alone. This increases the likelihood that some models in the candidate pool $\mathcal{M} = \{f_{\theta^{(1)}}, f_{\theta^{(2)}}, \dots, f_{\theta^{(N)}}\}$ will exploit dataset-specific noise or statistical flukes, resulting in overly optimistic validation scores.

Let $L_{i,t}$ denote the observed validation loss for run i at checkpoint t and decompose it as $L_{i,t} = v_{i,t} + Y_{i,t}$, where $v_{i,t} := \mathbb{E}[L_{i,t}]$ is the expected validation loss with respect to the underlying data distribution, and the zero-mean term $Y_{i,t}$ captures the random fluctuation due to finite-sample effects, so that $\mathbb{E}[Y_{i,t}] = 0$. Define the best (true) population validation loss as $v_\star := \min_{i \leq N, t \leq q} v_{i,t}$. Then

$$\begin{aligned} \mathbb{E} \left[\min_{i \leq N, t \leq q} L_{i,t} \right] &= \mathbb{E} \left[\min_{i,t} (v_{i,t} + Y_{i,t}) \right] \\ &\geq v_\star + \mathbb{E} \left[\min_{i,t} Y_{i,t} \right] = v_\star - \mathbb{E} \left[\max_{i,t} (-Y_{i,t}) \right]. \end{aligned}$$

If the centered noise $Y_{i,t}$ is sub-Gaussian with variance proxy v (scale $s_m^2 = v$) on $\mathcal{D}_V$, using the inequality for the expectation of a maximum [12, Sec. 2.5, p. 31], we have $\mathbb{E} \max_{j \leq M} Z_j \leq (2v \log M)^{1/2}$ for $M = Nq$. Applying this to $Z_j \equiv -Y_{i,t}$ yields

$$\mathbb{E} \left[\min_{i \leq N, t \leq q} L_{i,t} \right] \geq v_\star - s_m \sqrt{2 \log(Nq)}.$$

More generally, if $Y_{i,t}$ is subgamma with parameters (v, c) , [12, Sec. 2.5, Cor. 2.6] gives $\mathbb{E} \max_{j \leq M} Z_j \leq (2v \log M)^{1/2} + c \log M$. Hence

$$\mathbb{E} \left[\min_{i,t} L_{i,t} \right] \geq v_\star - \left(\sqrt{2v \log(Nq)} + c \log(Nq) \right).$$

The above derivation quantifies the expected optimism $v_\star - \mathbb{E}[\min_{i,t} L_{i,t}]$, capturing how much the best observed validation loss appears better than its true expected value. It grows at least on the order of $\mathcal{O}((\log(Nq))^{1/2})$, indicating that enlarging the search space (through more runs or checkpoints) inevitably amplifies VO.

The risk of VO is then further influenced during ensemble construction. Let $L_V(\mathcal{E})$ denote the validation loss of an ensemble $\mathcal{E}$ evaluated on $\mathcal{D}_V$. Suppose the ensemble is built incrementally by selecting, at iteration k

$$\mathcal{E}_k = \mathcal{E}_{k-1} \cup \{f_{\theta^{(k)}}\}, \quad f_{\theta^{(k)}} \in \arg \min_{f \in \mathcal{M} \setminus \mathcal{E}_{k-1}} L_V(\mathcal{E}_{k-1} \cup \{f\})$$

that is, by greedily adding the candidate that yields the largest reduction in validation loss. Each selection step thus introduces an additional layer of validation-driven optimization, effectively replacing $\mathcal{M}$ with a sequence of model subsets whose composition depends on repeated access to $\mathcal{D}_V$. Consequently, the ensemble selection trajectory $\{\mathcal{E}_k\}_{k=1}^K$ can

accumulate multiple opportunities for validation overfitting, since each incremental decision exploits random fluctuations in L_V in a manner analogous to the $\min_{i,t} L_{i,t}$ effect analyzed above. In expectation, this induces a compounded optimism bias that scales with both the ensemble size K and the effective search complexity of the candidate pool, thereby amplifying the gap between validation and test performance (TP) estimates.

To summarize, the overall model-induced VO risk is governed by the following three factors.

- 1) The number of validation checkpoints q per training run, which amplifies exposure to the validation set and increases the likelihood of overfit checkpoints.
- 2) The number of models N in the candidate pool, which controls the chance of including validation outliers.
- 3) The final ensemble size K , which governs how many potentially biased models are aggregated.

Together, these factors suggest that VO risk increases with the overall validation query budget ($N \times q$), and with the aggressiveness of validation-driven ensemble selection (K).

2) *Dataset-Induced VO Risk*: A small and unrepresentative validation set can lead to VO, especially if model developers prioritize validation performance as described before. This can intrinsically arise from inherent distributional differences between the validation and test sets, referred to also as “distributional-shift,” which is commonly seen in competition-driven or real-world environments [2], [3].

To formalize this, we analyze the test-time generalization risk in the presence of a class prior shift between validation and testing distributions. Let

$$\pi_V = (\pi_{V,1}, \dots, \pi_{V,C}) \quad \text{and} \quad \pi_E = (\pi_{E,1}, \dots, \pi_{E,C})$$

denote the class prior distributions in the validation and test sets, respectively, where C is the number of classes.

Following [2], let $R_V(f)$ and $R_E(f)$ be the expected risk of a classifier f on validation and test data:

$$\begin{aligned} R_V(f) &= \sum_{i=1}^C \pi_{V,i} \cdot \mathbb{E}_{x \sim \mathcal{D}_V(x|y=i)} [\ell(f(x), i)] \\ R_E(f) &= \sum_{i=1}^C \pi_{E,i} \cdot \mathbb{E}_{x \sim \mathcal{D}_E(x|y=i)} [\ell(f(x), i)]. \end{aligned}$$

Assuming conditional distributions $\mathcal{D}(x|y)$ are approximately stable (i.e., label shift setting), the test risk can be decomposed as follows:

$$\begin{aligned} R_E(f) &= R_V(f) + \sum_{i=1}^C (\pi_{E,i} - \pi_{V,i}) \\ &\quad \cdot \mathbb{E}_{x \sim \mathcal{D}(x|y=i)} [\ell(f(x), i)]. \end{aligned}$$

Bounding the per-class expected loss by a constant M , we obtain

$$\begin{aligned} |R_E(f) - R_V(f)| &\leq M \cdot \sum_{i=1}^C |\pi_{E,i} - \pi_{V,i}| \\ &= 2M \cdot \delta_{TV}(\pi_V, \pi_E) \end{aligned}$$

where δ_{TV} is the total variation distance between class priors.

This bound shows that the generalization risk grows proportionally with the magnitude of class prior shift. In other words,

the greater the distributional shift between the validation and test sets, the higher the risk of validation overfitting becomes.

To capture broader distributional shifts, we employ the JSD [9] between the model's predictive output distributions on the validation and test sets. Let P and Q denote the Softmax output distributions of a model f on validation and test sets, respectively. The JSD is defined as follows:

$$\text{JSD}(P \parallel Q) = \frac{1}{2}D_{\text{KL}}(P \parallel M) + \frac{1}{2}D_{\text{KL}}(Q \parallel M)$$

where $M = 1/2(P + Q)$ and $D_{\text{KL}}(P \parallel Q)$ is the Kullback–Leibler divergence

$$D_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}.$$

JSD is symmetric and bounded in $[0, 1]$, making it suitable for quantifying validation-to-test shift. A high JSD implies a strong distributional shift between the validation and test sets; thus, the risk of validation overfitting increases and the performance on $\mathcal{D}_V$ becomes an unreliable proxy for performance on $\mathcal{D}_E$.

This justifies the use of JSD in our VO-aware evaluation protocol. Specifically, we use JSD in two ways: 1) to construct validation/test splits with controlled shifts for benchmarking and 2) to assess VO robustness post hoc. High JSD coupled with small VO gaps and high test accuracy indicates a method's ability to generalize across significant shifts, a key property of VO-resilient pipelines.

B. Implications and Summary

Based on the previous analysis, we can conclude that validation overfitting is tightly linked to the following.

1) Ensemble-Modeling Perspective:

- a) The generation strategy followed to train individual base models, which is highly relying to the total number (N) of generated models.
 - b) The selection strategy followed to select the members (K) of the final ensemble predictor.
- Larger N increases the chance of selecting validation outliers, while larger K compounds noise accumulation during ensemble construction.

2) Dataset Perspective: Which relates to the inherent distributional-shift magnitude between training/validation and testing sets that further amplifies the risk of VO during model development.

Ensemble pipelines that heavily rely on validation feedback during model generation and selection are particularly vulnerable to these effects. Consequently, to mitigate validation overfitting in ensemble pipelines, it is crucial to: minimize the number of validation queries during base model generation, promote intrinsic diversity among models without heavy validation reliance, and reduce final ensemble size while maximizing representational diversity. These insights form the theoretical motivation behind the design of the proposed GeNeX framework, which aims to minimize validation reliance and promote representational diversity, to ensure robust generalization and minimize VO risk.

Moreover, to ensure rigorous benchmarking of VO robustness, evaluation protocols must explicitly simulate real-world

shifts. The use of the proposed JSD-guided partitioning allows for controllable distributional divergence between training/validation and testing sets, enabling researchers to create VO-prone scenarios by design in any case-study dataset.

Beyond split construction, in our framework, JSD is used as a post hoc indicator to assess dataset difficulty and VO risk. A model achieving high test accuracy and low VO gap under a high-JSD split provides compelling evidence of robustness and generalization. Conversely, performance in low-JSD scenarios (i.e., in-distribution) carries less evidential weight of the model's robustness against VO, as such conditions are inherently easier.

III. RELATED WORK

Having established the theoretical foundations and challenges of validation overfitting, we now examine existing research across three key areas: 1) standard ensemble learning strategies for robustification; 2) evolutionary and genetic-based approaches for promoting ensemble diversity; and 3) expert-aware and validation-resilient ensemble formulations.

A. Classical Ensemble Learning

Ensemble methods such as bagging, boosting, and snapshot ensembling are long-established techniques for improving model generalization and reducing variance [13], [14]. These approaches promote diversity through data resampling, stochastic training schedules, or architectural variation. While effective against TO, these strategies remain susceptible to validation overfitting, especially when model selection or ensemble formation relies heavily on validation score improvements [1].

Widely used incremental ensemble methods [4] progressively select models based on validation gains, aiming to iteratively improve validation accuracy. However, such strategies risk accumulating noise from validation fluctuations, particularly in low-data or high-variance settings, leading to validation overfitting [5].

B. Genetic and Evolutionary Ensemble Construction

To mitigate validation reliance and promote model diversity, population-based and genetic approaches have gained traction. Recent works have proposed hybrid frameworks combining deep learning with genetic algorithms for medical image analysis [7], [8], or for instance selection in ensembles [6]. These methods generate new models through crossover and mutation operations in weight or feature space, promoting exploration of under-represented regions in the model space. While effective in enhancing diversity and bypassing over-optimized local minima, they often stop short of formal VO mitigation during ensemble construction.

C. Expert-Aware and VO-Resilient Ensemble Formulations

A newer line of research focuses on explicitly managing behavioral diversity in ensembles through expert modeling and controlled generalization. In a recent work [2], the authors propose a controllable expert generation framework using hypernetworks to inject targeted diversity across long-tailed data regimes, demonstrating robustness under

domain distribution shifts. Similarly, CBDMoE [3] introduces a consistent-but-diverse mixture of experts architecture to improve domain generalization without sacrificing model consistency. These works highlight the importance of crafting ensembles from complementary submodels while balancing robustness and diversity. Complementary to the above, heterogeneity-driven expert ensemble methods [1], [5] have recently emerged. These methods cluster models by prediction behavior and select one representative expert per cluster using criteria like validation score or centroid proximity. While these strategies improve model diversity and mitigate VO to some extent, they remain vulnerable, since the initial model generation relies on validation-based monitoring, increasing the risk of noisy models infiltrating clusters, compromising the final ensemble's integrity.

D. Proposed Approach

In contrast to all previous works, the proposed GeNeX framework directly addresses validation overfitting at both model generation and ensemble construction/selection stages. During model generation, GeNeX entirely avoids the use of validation feedback by employing a dual-path strategy that combines gradient-based supervised training with genetic-based model evolution. This design is grounded in the hypothesis that generating offspring networks via crossover of gradient-trained parent models acts as a weight-level regeneration mechanism—mitigating overfitting while promoting structural diversity. As a result, GeNeX forms a candidate model pool that is both diverse and robust, without introducing validation-overfitted candidates.

At the ensemble construction stage, instead of selecting a single representative expert per cluster, GeNeX elects multiple diverse experts using complementary criteria and fuses them via weight-level averaging into robust prototype models. This prototype-based construction ensures representational richness and behavioral complementarity within the final ensemble. Overall, GeNeX achieves strong generalization with minimal reliance on validation signals, offering enhanced robustness in both limited-data and distributionally shifted scenarios.

IV. GENEX FRAMEWORK

In this section, we present the core design principles and components of the proposed GeNeX framework, detailing how its dual-module architecture aims to address validation overfitting by rethinking both model generation and ensemble construction. The implementation code is made available at the following link.¹

A. Overview of the GeNeX Framework

The GeNeX framework is designed to systematically address the challenge of VO by minimizing the dependency on validation feedback during model generation and ensemble construction. It consists of two main modules: a GenE constructor and a ProtoNeX selector.

The first module, GenE, aims to create a diverse and robust population of base networks through a hybrid training strategy.

In each iteration, networks are generated via two complementary paths: 1) supervised training with diverse configurations, under zero validation dependency; and 2) genetic weight fusion, where model weights are combined and mutated to generate new offspring networks. This genetic process aims to refresh potentially overfitted networks by encouraging exploration of the model space beyond gradient-based optimization, promoting diversity and robustness.

The second module, ProtoNeX, focuses on building a complementary ensemble from the generated network pool. Networks are clustered based on their predictive behavior, and multiple eXpert models are elected within each cluster based on diverse criteria, such as validation performance, robustness to perturbations, representativeness, and anomalous behavior. The selected experts within each cluster are then fused at the weight level to create a prototype model that encapsulates the cluster's representational diversity. Finally, all prototype models are combined into a final ensemble, with the ensemble weights optimized via sequential quadratic programming (SQP) [15] to maximize output-level synergy.

An abstract overview of the GeNeX pipeline is illustrated in Fig. 1, showcasing the main phases and the flow of model generation, clustering, expert selection, and prototype fusion. Through this multiphase design, GeNeX aims to generate validation-robust ensembles that maintain high generalization performance while systematically reducing the risk of validation overfitting.

1) GenE Module (GenE Generator): The first module of the GeNeX framework, referred to as GenE generator, aims to generate a diverse and robust pool of base networks $\mathcal{M}$ with minimal reliance on validation feedback. To achieve this, GenE integrates two complementary model generation paths: a *supervised* and *genetic-based*.

a) Supervised training path: In the supervised training path, a set of base networks is trained conventionally using labeled data and gradient-based optimization. To promote diversity within the population, various configurations are randomized across models, including different random seeds, data augmentations, dropout rates, learning rate schedules, and initialization schemes. This step ensures that even purely supervised models explore distinct regions of the model space.

Unlike standard deep learning pipelines that rely on validation monitoring, this phase deliberately avoids selecting best-validation checkpoints; models are trained for a fixed, small number of epochs (1–3).

b) Genetic-based path: Parallel to supervised training, GenE introduces a genetic-based model generation mechanism inspired by evolutionary algorithms. In this path, random pairs of independently trained parent networks are selected from the existing pool, and new child networks are generated via weight-level fusion and mutation. The genetic operations include the following.

Weight Fusion (Crossover): Given two parent networks, their corresponding weight tensors are combined through element-wise operations (e.g., averaging, interpolation, or randomized mask-based selection) to generate the child's initial weights. This operation allows knowledge from both parents to be inherited and mixed, potentially creating new representational patterns without relying on gradient flows.

¹GeNeX implementation link.

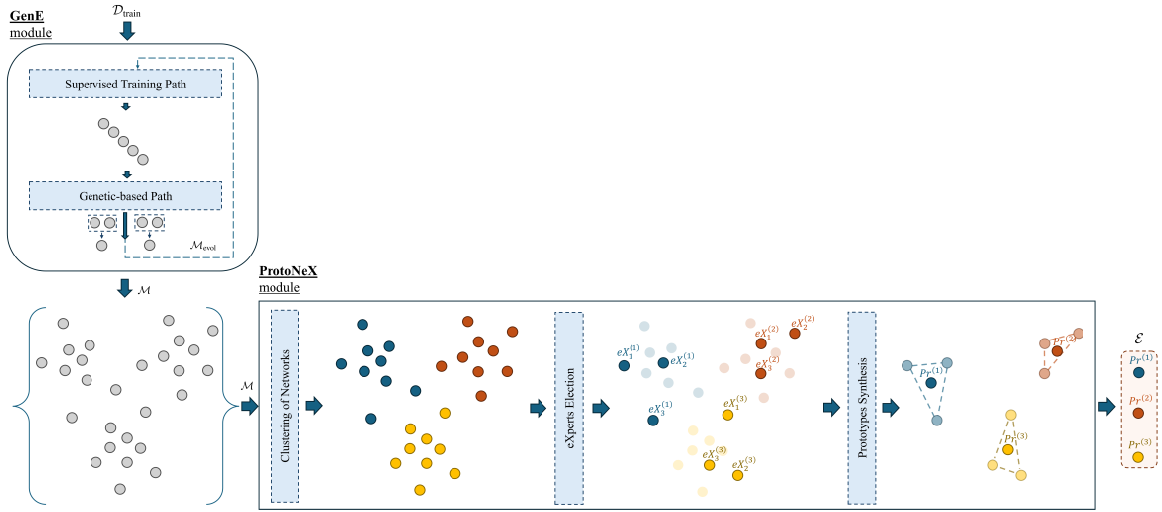


Fig. 1. GeNeX overview. GenE generates a diverse pool $\mathcal{M}$ without validation monitoring via short supervised training and genetic crossover/mutation, encouraging broad weight-space exploration and limiting early validation dependence. ProtoNeX clusters models in behavior (prediction) space, elects complementary experts via multicriteria selection, and fuses them into K compact prototypes. Instead of clustering data and training separate models per cluster, ProtoNeX clusters the models themselves and uses prototype fusion, promoting complementarity and behavioral diversity across the model space to enhance generalization.

Mutation: After fusion, controlled Gaussian noise is injected into random layers or parameters of the child network. Mutation encourages exploration of the weight space, introducing new variations that may lead to better generalization.

Children's networks generated through this genetic process are lightly fine-tuned (last task-specific layer) on training data for a small number of epochs, further maturing and adapting their representations without validation supervision.

Through iterative supervised training and genetic operations, the model population gradually expands, encompassing a wide variety of architectures, training trajectories, and weight configurations. Crucially, because model generation relies primarily on intrinsic diversity mechanisms rather than validation-based selection, the resulting population is less vulnerable to validation overfitting.

2) **ProtoNeX Module (Prototype-Based Network eXperts Selector):** After generating a diverse pool of base networks through the GenE module, the ProtoNeX module aims to construct a robust and complementary ensemble predictor $\mathcal{E}$, by systematically selecting and synthesizing representative expert models.

a) **Behavioral clustering of networks:** To organize the generated network pool, we first cluster the models based on their prediction behaviors. Specifically, models are embedded into a feature space derived from their output vectors on a shared validation subset, capturing their functional similarity. Clustering is then performed via Gaussian mixture models (GMMs) [16], grouping models into subpopulations that exhibit similar predictive patterns. This clustering step ensures that models specializing in different aspects of the data distribution are separated, facilitating complementary expert selection in subsequent steps.

b) **Expert election within clusters:** Within each cluster, multiple expert networks are elected based on diverse and orthogonal criteria, ensuring that various traits of model quality and behavior are captured: the top validation performer, the most robust under perturbations, the representative closest to the cluster centroid, the intraanomalous expert farthest from

it (capturing rare behaviors), and the outer-anomalous expert being most dissimilar to other clusters. By electing multiple experts with different selection criteria, ProtoNeX ensures that each cluster's representational space is thoroughly and reliably captured, reducing the risk of noisy or biased selections.

c) **Prototype synthesis via weight fusion:** Rather than selecting a single expert per cluster, ProtoNeX synthesizes a prototype model by fusing the internal weights of the elected experts. Weight fusion is performed via layer-wise averaging, aligning corresponding weights across models. This prototype model encapsulates the collective knowledge, robustness, and diversity of its cluster, offering a lightweight yet expressive representation without inflating model size. Finally, each prototype model is lightly fine-tuned (last task-specific layer) on training data, without validation supervision, to adapt their fused representations.

d) **Ensemble weight optimization:** Once all cluster prototypes are generated, they are combined into the final ensemble via SQP, yielding a convex combination of prototype predictions. Convex weighting can outperform simple averaging, especially when base models differ in reliability [17]. At the same time, restricting the combiner to a simplex-constrained convex combination gives a low-capacity model whose generalization properties are better controlled than those of unconstrained linear or more complex nonlinear stacking schemes [18]. In our case, the combiner introduces only a small number of simplex-constrained parameters (one weight per prototype), which has substantially lower capacity than typical nonlinear stacking models (e.g., neural-network meta-learners or higher-order polynomial combinations of base predictions), reducing the risk of overfitting the validation set.

Through this structured ensemble construction, ProtoNeX builds a final model that is both robust to VO and capable of strong generalization across diverse real-world tasks.

B. Theoretical View of GeNeX Methodology

The proposed GeNeX framework is designed to mitigate validation overfitting by enforcing diversity, complementarity,

and representational robustness at multiple stages of model generation and ensemble construction.

1) *Genetic Process as Diversity-Inducing and Overfitting-Resilient Path*: Let the model population be initialized as a set of N networks trained via supervised learning: $\mathcal{M} = \{f_{\theta^{(i)}}^{\text{grad}}\}_{i=1}^N$, where each model is trained with randomized configurations (e.g., seeds and augmentations) under gradient-based optimization. GenE avoids checkpoint-based validation monitoring during model generation to minimize validation overfitting. Under the same maximal-inequality view used in Section II-A1, the expected optimism from selecting among M validation queries scales as $\mathcal{O}((\log M)^{1/2})$; standard pipelines induce $M=Nq$ via N runs and q checkpoints, whereas GenE's generation stage contributes no such querying and any later selection is independent of q [at most $\mathcal{O}(N)$], effectively removing the $\log q$ term. However, via such an approach, classic TO remains a concern; here comes our hypothesis that a genetic-based process could refresh the population to mitigate training and validation overfitting while promoting diversity. Let $f_{\theta^{(j)}}^{\text{gen}}$ denote a child model generated by fusing weights of two parent models from $\mathcal{M}$: $\theta^{(j)} = \alpha\theta^{(a)} + (1 - \alpha)\theta^{(b)} + \epsilon$, where $\alpha \in [0, 1]$ controls crossover balance and $\epsilon \sim \mathcal{N}(0, \sigma^2)$ models mutation noise. Unlike gradient descent, GenE's operators act as geometric regularizers in weight space. Crossover and small-variance mutation moves offspring away from sharp parent minima toward broader, flatter valleys in the loss landscape. This mechanism is aligned with evidence that weight averaging and weight-space interpolation yield flatter optima and better generalization, thereby narrowing training/validation–test gaps [14], [19], [20].

Claim: Given two overfit parents $f_{\theta^{(a)}}$, $f_{\theta^{(b)}}$, their fusion under random crossover and mutation produces a model $\mathbb{E}[f_{\theta^{(j)}}^{\text{gen}}]$ that lies in a region less likely to inherit overfit features directly, acting as a diversity-preserving corrective path.

2) *Behavioral Clustering as a Complementarity Enforcer*: Let $\mathcal{M}$ be the full population of trained and genetically evolved models. For each model f_{θ} , we extract its prediction vector $z_{\theta} \in \mathbb{R}^d$ (e.g., concatenated Softmax outputs on a shared validation subset). To identify groups of behaviorally similar models, we cluster these prediction vectors using *GMMs*. Compared to hard-partitioning methods such as k -means, the GMM yields a probabilistic, soft clustering and allows anisotropic, overlapping clusters in high-dimensional spaces, which is well-suited for modeling correlated prediction patterns of neural networks [16], [21], [22]. This choice therefore provides a more expressive representation of behavioral similarity than common spherical-cluster or distance-only alternatives [22]. In this context, each cluster is modeled as a multivariate Gaussian distribution with its own mean μ_k and covariance matrix Σ_k . The assignment of a model to a cluster is based on the component that gives the highest likelihood under the Gaussian probability density function: $\text{Cluster}(z_{\theta}) = \arg \max_{k \in \{1, \dots, K\}} \mathcal{N}(z_{\theta}; \mu_k, \Sigma_k)$, where $\mathcal{N}(z_{\theta}; \mu_k, \Sigma_k)$ denotes the multivariate Gaussian density function: $1/((2\pi)^{d/2} |\Sigma_k|^{1/2}) \exp(-1/2(z_{\theta} - \mu_k)^T \Sigma_k^{-1} (z_{\theta} - \mu_k))$ and d is the dimensionality of the prediction vectors.

Each cluster $\mathcal{C}_k$ groups models with similar predictive characteristics, allowing us to assign distinct semantic roles or “expert behaviors” to different subpopulations. This enables:

intraclass specialization; within each $\mathcal{C}_k$, expert models may capture distinct patterns specific to a data subspace. Intercluster complementarity; across $\{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, clusters span diverse predictive behaviors, ensuring that the final ensemble captures a broader function space. This clustering step plays a critical role in organizing the model pool into behaviorally distinct groups, enabling the following expert election and prototype fusion stages to focus on complementary model traits from each cluster.

3) *Prototype-Based Weight Fusion as Representational Aggregation*: Within each cluster $\mathcal{C}_k$, a set of expert models $\{f_{\theta^{(k,1)}}, \dots, f_{\theta^{(k,m)}}\}$ is selected based on orthogonal criteria (e.g., top-validation, robust to noise, representative, and anomalous). These experts are fused into a prototype model via layer-wise averaging: $\theta^{(k)} = 1/m \sum_{j=1}^m \theta^{(k,j)}$. This operation aggregates both the dominant and diverse representational traits within each cluster. We define the prototype network $f_{\theta^{(k)}}$ as $f_{\theta^{(k)}} = \text{Prototype}(\mathcal{C}_k) = \arg \min_{\theta} \sum_{j=1}^m \|\theta - \theta^{(k,j)}\|^2$, that is, a centroid in weight space that minimizes intracluster variance.

Claim: The prototype network approximates the central representation of its cluster while preserving performance-relevant properties from its expert constituents.

Thus, combining K such prototypes, we form an ensemble: $\mathcal{E} = \sum_{k=1}^K w_k f_{\theta^{(k)}}$, with combination weights w_k optimized via SQP.

C. Algorithmic View of the GeNeX Framework

The GeNeX framework is composed of two sequential modules: 1) GenE, which generates a pool $\mathcal{M}$ of N diverse networks; and 2) ProtoNeX, which transforms $\mathcal{M}$ into a robust ensemble $\mathcal{E}$ predictor.

The *GenE* module (Algorithm 1) constructs a validation-agnostic ensemble model pool through an iterative evolutionary process designed to maximize diversity and minimize overfitting. It begins with a supervised initialization phase (Steps 5–10), where N models are trained independently using randomly sampled configurations from the configuration space $\mathcal{C}$. Each model is trained for a few epochs (e.g., 1–3) on the training set $\mathcal{D}_T$, without any validation monitoring or checkpointing, forming the initial evolutionary pool $\mathcal{M}_{\text{evol}}$. The core of GenE is a multigenerational loop (Steps 12–32), in which two complementary paths are executed.

Genetic Path (Steps 14–20): A set of N_g offspring models is generated via random pairwise weight fusion of existing models in $\mathcal{M}_{\text{evol}}$. For each pair of parents $f_{\theta^{(a)}}$, $f_{\theta^{(b)}}$, a child is formed by interpolating their weights using a convex combination with a crossover rate α and an additive Gaussian mutation term ($\theta^{(j)} = \alpha\theta^{(a)} + (1 - \alpha)\theta^{(b)} + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$). In all experiments, we fix $\alpha = 0.5$ (balanced inheritance), which is qualitatively robust (very small α reduces novelty, while $\alpha \approx 1$ drifts toward a single parent). The Gaussian mutation is applied layer-wise in convolutional/linear weight tensors (skipping buffers and batch-norm statistics) with probability ρ per tensor. We fix $\rho = 0.05$ and $\sigma = 0.01$ across datasets; larger (ρ, σ) increases exploration/diversity but can reduce stability, while smaller values under-explore. Mutation can be disabled by setting $\rho = 0$. Finally, the resulting child models are lightly fine-tuned (output head) on $\mathcal{D}_T$ to stabilize them postcrossover-and-mutation, and temporarily stored in $\mathcal{M}_{\text{gen}}$.

Algorithm 1 GenE Module: GenE Generation

Input: Training set $\mathcal{D}_T$, base configuration set $\mathcal{C}$, total target models N , number of generations G , genetic offspring per generation N_g

Output: Model pool $\mathcal{M}$ of size N

- 1: **Initialization**
- 2: Initialize output model pool $\mathcal{M} \leftarrow \emptyset$
- 3: Set per-generation selection quota $s \leftarrow \frac{N}{G}$
- 4: **Gradient-based Initialization (Generation 0)**
- 5: Initialize evolutionary model pool $\mathcal{M}_{\text{evol}} \leftarrow \emptyset$
- 6: **for** $i = 1$ to N **do**
- 7: Sample configuration $\mathcal{C}_i \in \mathcal{C}$
- 8: Train model $f_{\theta^{(i)}}$ on $\mathcal{D}_T$ (no validation monitoring)
- 9: Add model to pool: $\mathcal{M}_{\text{evol}} \leftarrow \mathcal{M}_{\text{evol}} \cup \{f_{\theta^{(i)}}^{\text{grad}}\}$
- 10: **end for**
- 11: **Evolution Loop**
- 12: **for** $g = 1$ to G **do**
- 13: *// Genetic Child Generation and Light Fine-tuning*
- 14: Initialize temporary set $\mathcal{M}_{\text{gen}} \leftarrow \emptyset$
- 15: **for** $j = 1$ to N_g **do**
- 16: Randomly select parents $f_{\theta^{(a)}}, f_{\theta^{(b)}} \in \mathcal{M}_{\text{evol}}$
- 17: Generate child weights via crossover and mutation:
 $\theta^{(j)} = \alpha\theta^{(a)} + (1 - \alpha)\theta^{(b)} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$
- 18: Lightly fine-tune $f_{\theta^{(j)}}$ on $\mathcal{D}_T$ (no validation monitoring)
- 19: Add to temporary pool: $\mathcal{M}_{\text{gen}} \leftarrow \mathcal{M}_{\text{gen}} \cup \{f_{\theta^{(j)}}^{\text{gen}}\}$
- 20: **end for**
- 21: Select randomly s models: $\mathcal{M}_{\text{sel}} \leftarrow \text{rand-}s(\mathcal{M}_{\text{gen}})$
- 22: Update output pool: $\mathcal{M} \leftarrow \mathcal{M} \cup \mathcal{M}_{\text{sel}}$
- 23: *// Gradient-based Training on Remaining Candidates*
- 24: Define $\mathcal{M}_{\text{rest}} = \mathcal{M}_{\text{gen}} \setminus \mathcal{M}_{\text{sel}}$
- 25: Add N_{new} fresh random initializations to $\mathcal{M}_{\text{rest}}$
- 26: Discard previous generation: $\mathcal{M}_{\text{evol}} \leftarrow \emptyset$
- 27: **for** each model $f_{\theta} \in \mathcal{M}_{\text{rest}}$ **do**
- 28: Sample new configuration $\mathcal{C}_k \in \mathcal{C}$
- 29: Train on $\mathcal{D}_T$ (no validation monitoring)
- 30: Update evolutionary pool: $\mathcal{M}_{\text{evol}} \leftarrow \mathcal{M}_{\text{evol}} \cup \{f_{\theta}^{\text{grad}}\}$
- 31: **end for**
- 32: **end for**

Selection and Gradient Path (Steps 21–31): A random subset of s models is selected from $\mathcal{M}_{\text{gen}}$ and added to the final output pool $\mathcal{M}$. The remaining $\mathcal{M}_{\text{rest}}$ children are forwarded to the supervised gradient path, where they are retrained under new randomly sampled configurations. Additionally, N_{new} freshly initialized models are introduced in each generation to further enhance diversity. These models, together with the nonselected children, form the next generation's population $\mathcal{M}_{\text{evol}}$. Over G generations, this procedure yields a final output pool $\mathcal{M}$ of exactly N models, each injected progressively across evolutionary stages. Importantly, with $s = N/G$ models added per generation, the final pool is composed of models from different evolutionary stages, promoting richer diversity in structural configurations and training trajectories. Crucially, the repeated injection of “freshly” initialized models throughout evolution prevents convergence to narrow regions of the weight space, further enhancing the robustness and complementarity of the resulting ensemble.

Algorithm 2 ProtoNeX Module: Prototype-Based Network eXperts Selection

Input: Model pool $\mathcal{M}$, training set $\mathcal{D}_T$, validation set $\mathcal{D}_V$, number of elected experts per cluster m

Output: Final ensemble model $\mathcal{E}$

- 1: **Behavioral Clustering**
- 2: Inference each $f_{\theta} \in \mathcal{M}$ on $\mathcal{D}_V$ to collect prediction vectors
- 3: Apply UMAP/GMM-based clustering on prediction vectors $\Rightarrow$ partition into K clusters $\{C_1, C_2, \dots, C_K\}$
- 4: **Expert Election per Cluster**
- 5: **for** each cluster C_k **do**
- 6: Elect m experts using diverse criteria: Top-Val Performer (max $\mathcal{A}_V$); Most Robust to perturbations; Cluster Representative (closest to centroid); Intra-anomalous (furthest from centroid); Outer-Anomalous (most distant from other clusters)
- 7: Denote selected experts as $E_k = \{f_{\theta_1}, \dots, f_{\theta_m}\}$
- 8: **end for**
- 9: **Prototype Synthesis via Weight Fusion**
- 10: **for** each E_k **do**
- 11: Fuse weights of all $f_{\theta} \in E_k$ via layer-wise averaging.
- 12: Lightly fine-tune output head on $\mathcal{D}_T$ without validation monitoring.
- 13: Add prototype $f_{\theta^{\text{proto}_k}}$ to ensemble set: $\mathcal{E} \leftarrow \mathcal{E} \cup \{f_{\theta^{\text{proto}_k}}\}$
- 14: **end for**
- 15: **Ensemble Weight Optimization**
- 16: Use SQP to find optimal weights $\{w_1, \dots, w_K\}$.

Algorithm 2 presents the core procedure of the *ProtoNeX* module, which transforms a genetically evolved diverse model pool into a compact and complementary ensemble by synthesizing prototype networks per behavioral cluster. The algorithm accepts as input the full model pool $\mathcal{M}$ (output from the preceding GenE stage), the training set $\mathcal{D}_T$, the validation set $\mathcal{D}_V$, and a user-specified number m of elected experts per cluster. The goal is to build a final ensemble $\mathcal{E}$ consisting of K prototype networks, one per behavioral cluster, each distilled from m diverse experts selected under orthogonal criteria.

1) *Behavioral Clustering (Steps 2 and 3):* Each model $f_{\theta} \in \mathcal{M}$ is fed on the validation set $\mathcal{D}_V$ to extract its class-probability Softmax vector, forming a high-dimensional prediction signature. These prediction vectors encode model behavior and are projected into a low-dimensional manifold using UMAP [23], capturing relative similarities among models based on their output distributions. Then, the GMM is applied in the latent space to group behaviorally similar models into clusters. The number of clusters K is automatically determined using silhouette score maximization. Each cluster C_k ideally represents a functionally distinct subbehavior (e.g., overconfident, conservative, robust-to-noise, etc.).

2) *Expert Election Per Cluster (Steps 5–8):* Within each cluster C_k , the algorithm elects m complementary experts that exhibit diverse behavior, measured via five criteria.

Top-Val Performer: Model achieving the highest validation accuracy within the cluster.

Most Robust: Model whose predictions remain the most stable under test-time input perturbations.

Cluster Representative: Model closest to the GMM cluster centroid, indicating typical behavior.

Intraanomalous: Model most distant from the cluster centroid, capturing edge-case behavior.

Outer-Anomalous: Model most dissimilar to all other clusters, acting as an outlier from a global standpoint. This expert election mechanism ensures that each cluster contributes not only its best-performing member but also diverse perspectives, including edge-case behaviors and stable predictors.

3) *Prototype Synthesis via Weight Fusion (Steps 10–14):* For each cluster, the weights of the selected m experts $\{\theta_1, \dots, \theta_m\}$ are averaged layer-wise to form a synthetic prototype θ^{proto_k} . This operation captures the common representational structure of the cluster and smooths out overfitted idiosyncrasies from individual experts. Each prototype is then lightly fine-tuned on the training set $\mathcal{D}_T$ to sharpen and adapt its output head. The refined prototype is added to the final ensemble set $\mathcal{E}$.

4) *Ensemble Weight Optimization (Step 16):* To integrate the K prototypes into a weighted ensemble, SQP is employed to optimize their scalar weights $\{w_1, \dots, w_K\}$, maximizing the ensemble's overall validation performance. This final step ensures that more contributive prototypes are weighted accordingly, resulting in a well-calibrated ensemble.

5) *Compute, Runtime, and Scalability:* GeNeX intentionally trades extra training-time exploration for low inference and memory overhead. In the GenE stage, the cost grows approximately linearly with the pool size N and the number of generations G and can be parallelized across models because candidates are trained for only a few epochs and offspring are adjusted with light fine-tuning. ProtoNeX clusters by prediction signatures and fuses one prototype per cluster; the deployed predictor contains K prototypes (typically 3–5), so inference is K forward passes (independent of N) and storage is K checkpoints. Empirically, for $N \in \{5, 500\}$, training time increases with N , while inference latency and memory remain essentially constant due to small K . Compared to classic ensembles that retain many models at test time, GeNeX concentrates computation in the offline training phase to gain behavioral diversity and minimize validation overfitting, while preserving fast and compact deployment.

V. CASE STUDY DATASETS

To evaluate the effectiveness of the proposed GeNeX framework in mitigating validation overfitting and improving generalization, we conducted experiments across four diverse challenging real-world classification tasks.

1) Skin Cancer Classification

Merging the datasets of the ISIC archive (thoroughly covering the period 2016–2024 [24], [25]), a unified skin lesion image dataset was constructed, containing approximately 13 000 samples per class (*benign*, *malignant*). This task reflects a critical real-world application in dermatological diagnosis and early skin cancer detection.

2) DeepFake Detection

A composite facial image dataset was constructed by merging three sources: the DeepFake Detection Challenge dataset

[26], DeepFakeMnist+ [27], and EXPerts (a dataset curated by professional Photoshop designers) [28]. The resulting dataset contains around 80 000 instances for the *fake* class and 18 000 for the *real* one. For each instance, facial regions were automatically detected and cropped from video frames, following protocols introduced in [29]. This task reflects real-world concerns in digital media forensics and misinformation detection.

3) Plant Disease Recognition

A curated plant disease dataset merged from 14 publicly available sources [30] was utilized, combining both laboratory and field images to ensure diversity in acquisition conditions. The final version includes approximately 57 000 samples for *diseased* and 20 000 for *healthy* class. This task reflects practical agricultural challenges in crop health monitoring and early disease detection.

4) Pneumonia Detection

The Chest X-ray dataset [31] was utilized, which contains 5856 validated chest radiographs from pediatric patients aged 1–5 years. Images were initially categorized into three labels: *normal*, *bacterial pneumonia*, and *viral pneumonia*. For our benchmark, we formulated a binary classification task by grouping all pneumonia types (4200 cases) as *pneumonia* and 1500 *normal* cases. This task reflects a high-impact real-world application in pediatric respiratory disease diagnosis using medical imaging.

VI. VO-AWARE EVALUATION PROTOCOL (RATIO-CONSTRAINED JSD-GUIDED CLUSTERING)

The proposed evaluation protocol simulates a limited-data regime, where the training set comprises only a small portion of the total data (e.g., 30%), and exhibits a strong distributional shift from the remaining 70% testing set. This setting reflects real-world and competition-style environments where such imbalances and shifts frequently occur [2], [3].

To simulate this, we introduce a ratio-constrained, JSD-guided partitioning method, which generates maximally divergent training/testing splits. Rather than performing random sampling, our method explicitly optimizes the separation between distributions of the two subsets, quantified by the JSD, while strictly preserving a user-defined class-wise ratio (e.g., 30–70). Importantly, to prevent interclass drift and avoid biasing the dataset class distribution, the procedure is applied independently for each class.

Let $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^N$ be the dataset with Softmax vectors $x_i \in \mathbb{R}^C$ and class labels $y_i \in \mathcal{Y}$. For each class $c \in \mathcal{Y}$, define $\mathcal{X}_c = \{x_i \mid y_i = c\}$ with $N_c = |\mathcal{X}_c|$. We seek a partition of $\mathcal{X}_c$ into disjoint subsets $\mathcal{X}_T^c$ and $\mathcal{X}_E^c$ of fixed ratio r such that the JSD between their distributions is maximized

$$\mathcal{X}_T^c, \mathcal{X}_E^c = \arg \max_{\substack{|\mathcal{X}_T^c| = rN_c \\ |\mathcal{X}_E^c| = (1-r)N_c}} \text{JSD}(\mu_T^c \parallel \mu_E^c)$$

where $\mu_T^c = 1/(|\mathcal{X}_T^c|) \sum_{x_i \in \mathcal{X}_T^c} x_i$ and $\mu_E^c = 1/(|\mathcal{X}_E^c|) \sum_{x_i \in \mathcal{X}_E^c} x_i$ are the mean Softmax vectors per split. The final global partition is given by $\mathcal{X}_T = \bigcup_{c \in \mathcal{Y}} \mathcal{X}_T^c$, and $\mathcal{X}_E = \bigcup_{c \in \mathcal{Y}} \mathcal{X}_E^c$.

This optimization problem defines the underlying objective of our partitioning scheme. To approximate this solution

Algorithm 3 Ratio-Constrained JSD-Guided Clustering

Input: Softmax vectors $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^N$; target ratio r ; maximum iterations T ; optional overlap $\zeta \in [0, 0.5]$ (default = 0)

Output: Training set indices $\mathcal{I}_T$; Testing set indices $\mathcal{I}_E$

```

1: Initialize  $\mathcal{I}_T \leftarrow \emptyset$ ,  $\mathcal{I}_E \leftarrow \emptyset$ 
2: for all class  $c \in \mathcal{Y}$  do
3:   Let  $\mathcal{X}_c = \{x_i \mid y_i = c\}$ , total  $N_c = |\mathcal{X}_c|$ 
4:   Randomly select  $r \cdot N_c$  samples as initial  $\mathcal{I}_T^c$  and the rest as  $\mathcal{I}_E^c$ 
5:   for iteration = 1 to  $T$  do
6:      $\mu_T^c \leftarrow \text{mean}(\mathcal{X}_{\mathcal{I}_T^c}^c)$ ,  $\mu_E^c \leftarrow \text{mean}(\mathcal{X}_{\mathcal{I}_E^c}^c)$ 
7:     for all  $x_i \in \mathcal{X}_c$  do
8:        $d_T(x_i) \leftarrow \text{JSD}(x_i, \mu_T^c)$ ;  $d_E(x_i) \leftarrow \text{JSD}(x_i, \mu_E^c)$ ;
        $\Delta_i \leftarrow d_E(x_i) - d_T(x_i)$ 
9:     end for
10:    Sort  $x_i$  by  $\Delta_i$  and reassign top  $r \cdot N_c$  to  $\mathcal{I}_T^c$ , rest to  $\mathcal{I}_E^c$ 
11:    if global JSD gain  $< \epsilon$  break
12:  end for
13:  if  $\zeta > 0$  then
14:     $s_c \leftarrow \lfloor \zeta \cdot |\mathcal{I}_T^c| \rfloor$ ; sample  $S_T^c \subset \mathcal{I}_T^c$ ,  $|S_T^c| = s_c$ ; sample  $S_E^c \subset \mathcal{I}_E^c$ ,  $|S_E^c| = s_c$ 
15:     $\mathcal{I}_T^c \leftarrow (\mathcal{I}_T^c \setminus S_T^c) \cup S_E^c$ ;  $\mathcal{I}_E^c \leftarrow (\mathcal{I}_E^c \setminus S_E^c) \cup S_T^c$ 
16:  end if
17:  Append  $\mathcal{I}_T^c$  to  $\mathcal{I}_T$ , and  $\mathcal{I}_E^c$  to  $\mathcal{I}_E$ 
18: end for

```

in practice, we employ an iterative refinement strategy that incrementally reassigns data points based on their relative divergence to the evolving training and test centroids. The complete procedure is detailed in Algorithm 3.

The input to the algorithm consists of Softmax vector representations of all data instances. These representations are extracted via a fixed, unbiased encoder Z , such as a frozen ImageNet-pretrained network, to ensure that no task-specific adaptation influences the embeddings. In addition, the input parameter “target ratio r ” controls the fraction of samples assigned to the training subset per class (e.g., $r = 0.3$ corresponds to 30% training and 70% testing sets ratios). This parameter reflects the severity of the limited data condition. The lower the training ratio, the more severe the data scarcity becomes, thereby amplifying the risk of validation overfitting.

The algorithm proceeds by iterating over each class $c \in \mathcal{Y}$ individually. In Step 3, for each class, the subset $\mathcal{X}_c$ of all instances belonging to that class is extracted. In Step 4, an initial random split (RS) is created within $\mathcal{X}_c$, assigning exactly $r \cdot N_c$ samples to the training subset $\mathcal{I}_T^c$ and the remaining $(1 - r) \cdot N_c$ to the test subset $\mathcal{I}_E^c$, where N_c is the number of instances in subset $\mathcal{X}_c$.

The main body of the algorithm (Steps 5–12) is an iterative refinement loop, executed for a fixed number of iterations T or until convergence. In each iteration, the algorithm in Step 6 first computes the mean Softmax vectors of the current training and test subsets, denoted by μ_T^c and μ_E^c , respectively. These centroids serve as reference points for evaluating how well each instance aligns distributionally with either subset. Next,

in Steps 7–9, for every instance $x_i \in \mathcal{X}_c$, two JSD values are computed: the divergence $d_T(x_i)$ between x_i and the training centroid μ_T^c , and the divergence $d_E(x_i)$ between x_i and the test centroid μ_E^c . The difference $\Delta_i = d_E(x_i) - d_T(x_i)$ is used to quantify the relative affinity of x_i toward the training or testing distribution. A positive Δ_i implies that x_i is closer to the training centroid than to the test centroid, while a negative value suggests the opposite.

Once all Δ_i values are computed, the algorithm in Step 10 ranks all instances in $\mathcal{X}_c$ based on their Δ_i scores. The top $r \cdot N_c$ instances with the highest Δ_i are reassigned to the training subset $\mathcal{I}_T^c$, and the remainder to the test subset $\mathcal{I}_E^c$. This reassignment attempts to maximize the separation between the two subsets by moving instances that are more aligned with the training centroid into the training set, and vice versa.

After each iteration, the algorithm in Step 11 checks for convergence by evaluating the change in the global JSD between the updated training and test centroids. This serves as a global divergence score indicating how different the two distributions have become. Importantly, this global JSD must be distinguished from the per-instance JSDs used during reassignment. While the per-instance values guide the local partitioning decisions, the global JSD quantifies the overall quality of the split. If the increase in global JSD between iterations falls below a small threshold ϵ , the process is considered converged, and the loop terminates early.

After the main ratio-constrained phase, which typically yields the maximum achievable JSD at the fixed per-class training/testing ratio, we add a single-parameter refinement that allows users to tune the desired shift level (Steps 13–16). Given an overlap ratio $\zeta \in [0, 0.5]$, we randomly sample $s_c = \lfloor \zeta \cdot |\mathcal{I}_T^c| \rfloor$ samples from the minority split and s_c samples from the majority split for each class and swap them. This preserves both the class balance and the training/testing ratio while monotonically reducing the global JSD as ζ increases. Users may vary ζ (e.g., 0.1, 0.2, 0.3) to control the shift strength. For instance, with $|\mathcal{I}_T^c| = 3000$, $|\mathcal{I}_E^c| = 7000$, and $\zeta = 0.2$, we swap 600 randomly sampled items in each direction. As an illustrative example for the Skin-Cancer dataset, with a target ratio $r = 0.1$, this refinement decreases the total JSD from 0.445 (at $\zeta = 0$) to 0.316, 0.229, 0.163, 0.110, and 0.069 at $\zeta \in 0.1, 0.2, 0.3, 0.4, 0.5$, respectively.

Finally, the per-class training and test indices are aggregated into global index sets (Step 17). The algorithm returns these indices, which can then be used to construct the actual data splits. It is worth emphasizing that this algorithm converges extremely fast in practice, typically requiring only 2–3 iterations per class to converge, while producing significantly divergent splits that are highly suitable for VO-aware generalization stress testing. Fig. 2 visually illustrates the resulting partitioning, where training and testing subsets exhibit clearly distinct intraclass patterns, demonstrating the algorithm’s effectiveness in partitioning data into distributionally divergent subsets. The full implementation of this algorithm is provided at the following link.²

This ability to enforce measurable distributional shift makes the proposed method especially useful for crafting stress-test benchmarks for evaluating generalization and VO behavior.

²Ratio-constrained JSD-guided clustering.

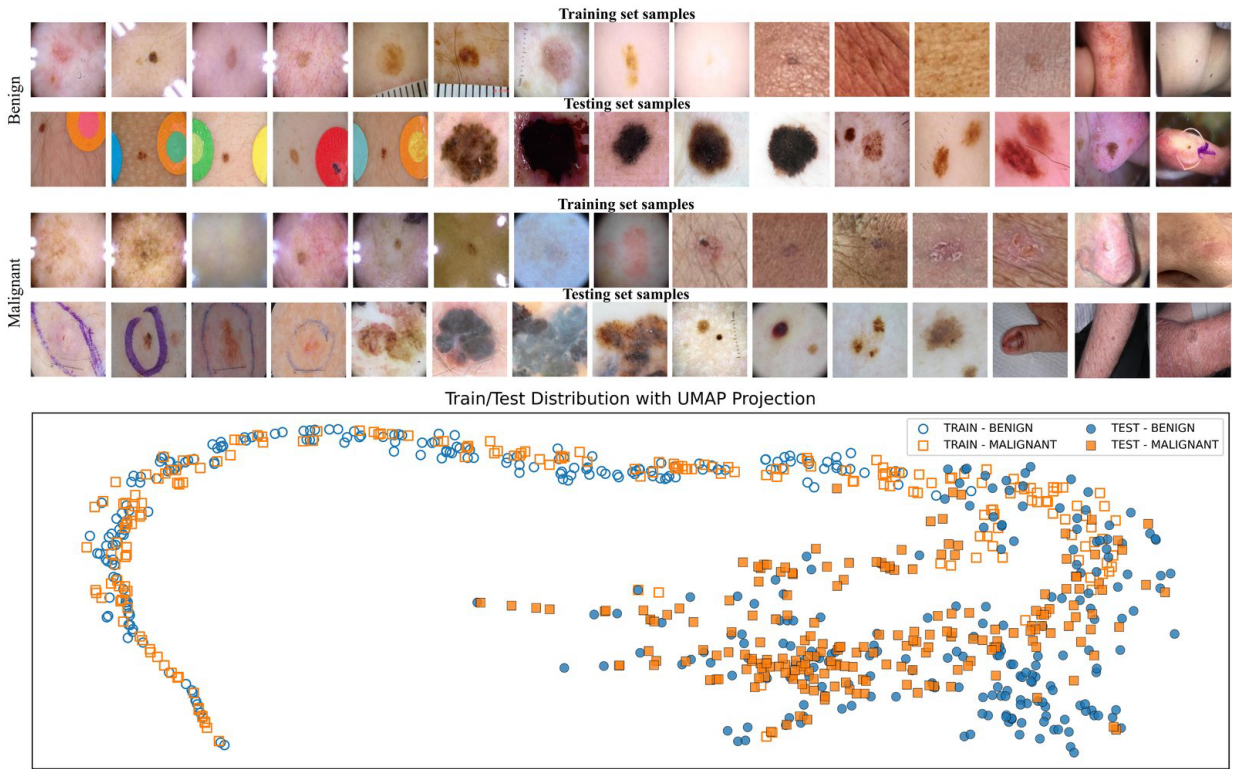


Fig. 2. Distribution-shifted training/testing partitioning based on the proposed JSD-guided clustering algorithm applied on the Skin-Cancer dataset. The goal of the method is to partition a dataset into training and testing subsets such that the JSD between their distributions is maximized. This process generates distributionally shifted splits that simulate realistic and challenging learning scenarios. Such splits are useful for reliably benchmarking a model's robustness to validation overfitting. Visual inspection reveals distinct data characteristics across the two subsets, emphasizing the diversity of the resulting partitions.

VII. EXPERIMENTAL ANALYSIS

This section presents the experimental setup and evaluation results of the proposed GeNeX framework across four challenging real-world binary classification tasks: Skin Cancer detection, DeepFake detection, Plant Disease recognition, and Pneumonia classification. We benchmarked GeNeX against a diverse set of baseline ensemble strategies under the proposed VO-aware evaluation protocol that deliberately enforces distributional shift to simulate realistic generalization challenges.

A. SoA Baselines Comparison Study

Table I provides a structured summary of all baseline methods and ensemble configurations involved in the experimental comparison. Each framework is characterized by its model generation strategy, the presence or absence of validation monitoring, the total number of generated models (N), the ensemble selection criteria, and the number of final models used for prediction (K). We compared GeNeX against the following approaches.

Single Model Selection [4]: Model selection from a pool of N trained networks based on top validation performance. The pool is generated via configuration-driven diversity search, wherein multiple models are trained under varying configurations and the top-performing one on validation is selected.

Incremental Ensemble (Inc-Ens) Construction [4]: A sequential ensemble construction strategy that progressively builds an ensemble by selecting K models from a pool of N diverse trained models, with each addition guided by improvements in validation performance.

Genetic-PneuNet [7]: A GA-based ensemble model originally designed for pneumonia classification. Models are evolved via crossover and mutation, and the ensemble is formed by selecting the best-performing offspring based on validation.

Genetic-SeqSel [6]: A sequential instance selection framework using GAs to iteratively build robust ensembles. Though focused on instance selection, the strategy emphasizes validation-guided evolution of ensemble members.

Genetic-BUSNet [8]: Combines deep learning and GAs for feature selection and ensemble classification in breast ultrasound image analysis. Emphasizes diversity in the model space with validation-driven selection.

CBDMoE [3]: A consistent-but-diverse learning (CBDL)-based MoE framework for domain generalization, where experts are trained with consistency and diversity losses, and validation-guided fusion enhances generalizability.

PRL [2]: A preference-vector (PV)-driven hypernetwork framework that generates diverse experts for long-tailed learning, enabling controllable head-tail tradeoffs and robust adaptation to testing distribution shifts.

Heterogeneity-Based eXperts (HeX) [1], [5]: Clusters models by prediction behavior and selects one expert per cluster using different criteria: *HeX-TV* chooses the top-validation performer within each cluster, while *HeX-C* selects the model closest to the cluster centroid in prediction space.

RSEN [32]: A robust self-ensembling network that maintains a teacher as the exponential moving average of a supervised student and regularizes training with a consistency loss between the teacher and the student under stochastic

TABLE I
SUMMARY CLASSIFICATION OF ALL UTILIZED FRAMEWORKS IN THE EXPERIMENTAL COMPARISON

Frameworks	Description	Generation		Selection	
		Validation Monitoring	N	Criterion	K
Single (1-ep)	Single model just trained for 1 epoch.	✗	1	-	1
Aver-Ens (1-ep)	Averaging ensemble of random initialized models trained for 1 epochs.		3		3
Single (N_{50})	(1) A pool of N models is trained under diverse configurations using a standard train-validation development pipeline.	✓	50	Top-Val-based	1
Single (N_{500})	(2) Validation performance is monitored to guide model development.		500		
	(3) The top-validation model is selected as final single predictor				
Inc-Ens (N_{50}, K_3)	Ensemble approach that generates N models as in (1) and (2).		50		3
Inc-Ens (N_{500}, K_3)	Incrementally selects K total models from the generated pool,		500		10
Inc-Ens (N_{500}, K_{10})	each chosen to progressively enhance validation performance.		500		
Genetic-PneuNet	Evolves models via GA and selects best offsprings by validation.		50	CBDL-based PV-based	3
Genetic-SeqSel	Builds ensembles by GA-driven, validation-based model selection				10
Genetic-BUSNet	Combines GA feature/model selection with validation-based ensemble.				4
CBDMoE	Trains experts with consistency-diversity loss for domain generalization.				
PRL	Uses hypernetworks to generate experts and adapt via preference vectors.				
HeX-TV (N_{50})	Performs behavioral clustering on the N generated models (1), (2).		50	Top-Val-based	Auto
HeX-TV (N_{500})	One expert per cluster is selected based on validation performance.		500		
HeX-C (N_{50})	Selects the model closest to the cluster centroid in prediction space,		50	Centroid-based	
HeX-C (N_{500})	promoting representativeness over raw performance within each cluster.		500		
RSEN	Moving-average teacher with consistency-filtered self-ensembling.	✗	Auto	-	1
HWA	Online averaging of parallel models with offline slide-window averaging.		2–4		
GeNeX (N_{50})	Fuses genetically evolved expert models per cluster		50	Prototype-based	3–5
GeNeX (N_{500})	into robust prototypes.	500			

augmentations. The moving-average ensemble network is used as the final predictor; adapted here to 2-D convs.

HWA [33]: A hierarchical weight-averaging scheme that periodically averages multiple models trained in parallel (online) and then applies a sliding-window average over saved outer checkpoints (offline), yielding a single robust predictor.

GeNeX differs from the above in that it utilizes validation-independent genetic-based model generation with cluster-wise prototype fusion, aiming to minimize validation reliance both during model generation and ensemble construction.

1) Experimental Settings: All base models across methods were trained using the Adam optimizer (initial learning rate 10^{-4}), with reduce-on-plateau scheduling, early stopping (patience = 5), and weight decay (10^{-4}) [34], [35]. For a fair comparison across all methods, we utilized the MobileNetV3-Small pretrained on ImageNet, selected for its strong accuracy–speed tradeoff and its effectiveness as a lightweight base learner for ensemble learning [36]. Input images were uniformly resized to 224×224 following standard CNN preprocessing practices [37]. The evaluation metric across all experiments was the geometric mean (GM), computed as $(\text{sensitivity} \times \text{specificity})^{1/2}$. GM offers a balanced assessment of classification performance across classes, especially under imbalanced settings [38]. All experiments were implemented in PyTorch and executed on NVIDIA RTX 3090 GPUs.

To quantify validation overfitting, we computed the VO gap for each model as the difference between validation and, using the same GM metric. For fairness, all methods used a consistent validation split (30% of the training pool) and were evaluated against the fixed shared test set. Table II reports the TP and VO gap for each framework across the four benchmark tasks. TP values correspond to GM performance, and VO gap is the GM difference (val-test), both scaled by 10^3 . The evaluation was performed under two distinct partitioning

strategies: 1) RS, where the training–testing split is created via classic stratified random sampling, resulting in near-zero JSD and thus low VO risk; and 2) guided split (GS) with maximized distributional shift (e.g., $\text{JSD} \in [0.1, 0.6]$ depending on the dataset) via the proposed JSD-guided partitioning algorithm (described in Section VI).

At this point, it is essential to clarify that our JSD splitter is a benchmarking utility designed to simulate shifted test environments for evaluating VO-resilience across methods. Using a ratio-constrained partitioning procedure, we construct a *train/validation/(shifted) test* split from the same labeled corpus, preserving fixed per-class ratios while maximizing the distributional separation between train/validation and (shifted) test. The resulting “test” set is a proxy used solely for VO-aware evaluation; not a production holdout from the deployment distribution.

The GeNeX framework is evaluated with two GenE population sizes, $N = 50$ and $N = 500$, where N denotes the number of models generated by the GenE module, to assess robustness under small and large candidate populations.

2) Discussion of Results: The experimental results across all four case study tasks confirm the effectiveness of the proposed GeNeX framework in mitigating validation overfitting and enhancing generalization. In RS settings ($\text{JSD} \approx 0$), where the validation and testing distributions are nearly identical, most methods perform comparably well, with minimal VO gaps. This is expected, as the absence of a distributional shift reduces the risk of overfitting, making generalization straightforward. However, in the GS conditions, where JSD is deliberately maximized to simulate realistic distributional shifts, most models that performed well in RS settings now experience a significant drop in performance. Interestingly, the 1-epoch baselines exhibit strong robustness, consistently yielding low VO gaps and competitive TP, particularly in the most challenging high-JSD tasks ($\text{JSD} > 0.3$). This indicates that

TABLE II

PERFORMANCE RESULTS UNDER THE PROPOSED VO-AWARE EVALUATION PROTOCOL: RATIO-CONSTRAINED JSD-GUIDED CLUSTERING ALGORITHM WAS APPLIED TO GENERATE HIGHLY DIVERGENT TRAINING-TESTING SPLITS (TARGET RATIO $r = 0.3$)

Frameworks	Skin Cancer				DeepFake				Plant Disease				Pneumonia			
	RS		GS		RS		GS		RS		GS		RS		GS	
	JSD = 1e-4		JSD = 0.360		JSD = 1e-4		JSD = 0.545		JSD = 1e-4		JSD = 0.434		JSD = 1e-4		JSD = 0.142	
	TP	VO	TP	VO	TP	VO	TP	VO	TP	VO	TP	VO	TP	VO	TP	VO
Single (1-ep)	696	<1	672	54	658	<1	585	19	769	<1	597	171	526	<1	533	120
Aver-Ens (1-ep)	702	<1	693	55	697	<1	605	16	847	<1	600	175	541	<1	645	113
Single (N_{50})	791	2	607	214	706	3	507	234	943	<1	533	380	914	<1	812	152
Single (N_{500})	802	8	577	248	715	9	445	298	946	2	540	395	929	<1	798	170
Inc-Ens (N_{50}, k_3)	813	13	623	236	741	11	429	334	949	5	504	434	932	<1	806	164
Inc-Ens (N_{500}, k_3)	825	15	597	267	752	17	401	397	952	8	498	443	946	<1	793	196
Inc-Ens (N_{500}, k_{10})	810	38	584	295	760	36	404	446	942	27	405	573	963	3	711	288
Genetic-PneuNet	814	7	659	142	771	8	556	157	950	2	574	346	962	<1	826	138
Genetic-SeqSel	821	12	646	162	768	9	571	175	953	3	566	379	952	<1	823	139
Genetic-BUSNet	819	8	683	101	777	9	591	181	962	1	558	383	955	<1	819	156
CBDMoE	815	3	696	99	772	4	567	80	954	<1	641	307	958	<1	830	148
PRL	809	2	701	87	793	1	617	52	969	<1	667	203	950	<1	838	112
HeX-TV (N_{50})	831	9	675	192	790	5	615	155	967	3	583	367	959	<1	817	170
HeX-TV (N_{500})	822	20	634	242	779	21	572	244	972	6	552	411	961	1	812	182
HeX-C (N_{50})	810	8	698	98	786	5	595	45	965	<1	628	284	953	<1	840	103
HeX-C (N_{500})	817	10	660	167	785	8	576	123	963	2	607	339	954	<1	831	106
RSEN	806	<1	721	39	780	<1	622	18	948	<1	699	175	956	<1	847	102
HWA	812	<1	706	45	783	<1	631	15	956	<1	702	172	943	<1	856	95
GeNeX (N_{50})	816	<1	718	46	788	<1	643	13	947	<1	705	167	947	<1	861	87
GeNeX (N_{500})	823	<1	716	41	784	<1	639	17	949	<1	698	183	951	<1	852	105

extremely short training, without validation monitoring, acts as a natural regularizer, limiting overfitting and enhancing robustness to distributional shift. GeNeX outperforms all baseline frameworks under GS settings in both TP and VO gap. This highlights its robustness in scenarios where validation-based model selection becomes unreliable. Notably, even under large model generation budgets (e.g., $N = 500$), GeNeX maintains minimal VO gaps, confirming that its validation-independent generation phase (GenE) and ProtoNeX effectively prevent validation-induced overfitting. In contrast, Inc-Ens and top-validation HeX variants show increasing VO gaps as N and K scale up. While their performance is strong in RS, these methods collapse under high JSD conditions in the GS setup. This aligns with the theoretical formulation in Section II, where we showed that validation-driven model search and ensemble expansion magnify VO risk, which is further amplified in high-JSD regimes.

Among the SoA baselines, genetic strategies like Genetic-PneuNet and Genetic-SeqSel offer improved VO resistance but still suffer under GS due to partial reliance on validation during selection. Likewise, CBDMoE and PRL demonstrate strong accuracy but display moderate VO gaps, particularly in the most shifted scenarios, highlighting the limitations of diversity-driven designs without validation control. HWA maintains low VO and stable generalization in both RS and GS. Its validation-independent hierarchical averaging acts as a strong regularizer, mitigating shift-induced variance without relying on validation-driven search. RSEN leverages self-ensembling regularization to temper validation overfitting and narrow VO gaps under shift. It remains competitive across settings, though its gains are generally milder than GeNeX in the most shifted regimes.

Overall, GeNeX provides robust performance and minimal VO gaps across all settings, outperforming all other methods in the high-risk GS conditions. Its resilience to validation-induced overfitting, without compromising TP, establishes it as a reliable framework for real-world deployments where validation signals are noisy, biased, or unrepresentative.

3) *Ablations and Statistics*: We conduct three studies to assess statistical significance, component utility, and hyperparameter robustness. We first apply a nonparametric Friedman aligned-ranks test [39] of the null hypothesis H_0 (equal TP across methods); if H_0 is rejected (meaning that the performance differences are not due to random variation), we run Finner post hoc test comparisons at significance level $\alpha = 5\%$ to examine the existence of significant differences. Next, we perform a ProtoNeX leave-one-out ablation over its expert-selection criteria to quantify each criterion's contribution. Finally, we sweep GenE's mutation probability ρ and noise scale σ to probe sensitivity. Full protocols and tables appear in the Supplement.³ The statistical test rejects H_0 and post hoc results show GeNeX to consistently surpass the strongest baselines. In the ProtoNeX ablation, removing any single criterion increases VO and lowers TP, with the largest drop occurring when the robust-to-perturbations criterion is omitted. In GenE, performance is stable for $\rho \in [0.02, 0.10]$ and $\sigma \in [5 \times 10^{-3}, 10^{-2}]$; very small values reduce diversity, whereas larger values can destabilize offspring.

³Supplementary Material (*supplementary.pdf*): Sections S1–S3, Tables S1–S3.

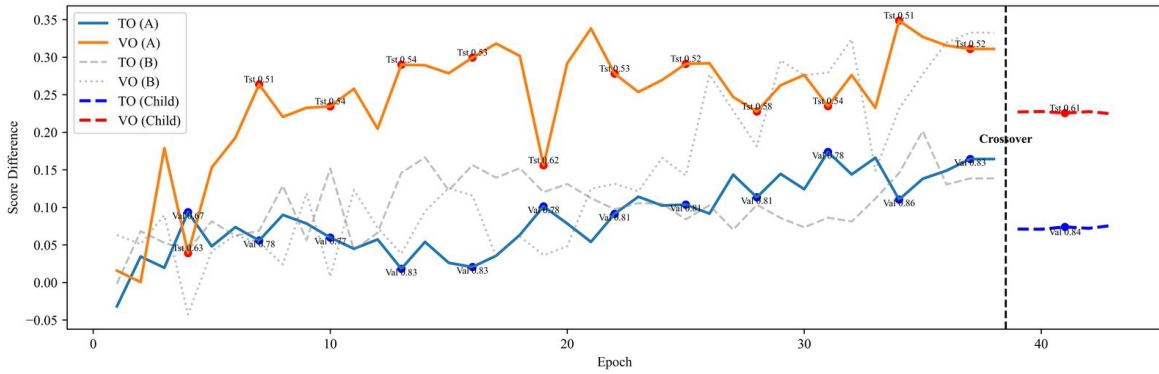


Fig. 3. Visualization of TO and VO gaps for two parent networks A and B and their genetically evolved child models. The study is conducted on the most challenging VO-pruned dataset, GS-DeepFake, which exhibits the highest distributional shift ($JSD = 0.545$). We observe that despite minimal TO and high validation performance (epochs 12–18 for A), severe VO gaps can still occur, leading to unreliable selection. After genetic crossover (vertical line), child models demonstrate improved generalization and reduced overfitting.

B. Inner Study: Genetic Process as an Overfitting-Resilient Path

To better understand the internal behavior of GeNeX, we conduct an in-depth study on the GS-DeepFake task, our most challenging VO-pruned benchmark with a pronounced distributional shift ($JSD = 0.545$). This inner-view analysis focuses on how genetic model generation mitigates both training and validation overfitting. Specifically, we investigate how two overfitted parent models, A and B (Fig. 3), evolve into more generalizable child models through genetic operations. To simulate the effect of overfitting, we deliberately train parent networks A and B for extended epochs, exposing them to both high TO and VO divergence. Then, apply weight-level crossover and mutation to generate child networks, followed by light fine-tuning limited only to the classification head. We hypothesize that this genetic crossover process acts as a weight-space regularizer, reinitializing over-optimized trajectories and promoting the exploration of new parameter regions with improved generalization. While TO can be visible during training and typically correlates with poor generalization, VO is more deceptive.

We observe that models exhibiting minimal TO and high validation scores (typically considered promising checkpoints) can still fail dramatically at test time. For instance, in the 12–18 epoch area for A, validation accuracy appears stable and strong, yet large VO gaps exist, highlighting the risk of relying solely on validation performance under distributional shift. After the crossover point (dashed vertical line), we evaluate the generated offspring/child model. Despite being derived from heavily overfitted parents, the child model consistently shows smaller TO and VO gaps and stronger TP.

Overall, this analysis reinforces our claim (Section IV-B1) that genetic operations within GenE serve as a recovery mechanism, which provides an implicit regularization effect, mitigating both TO and VO.

VIII. CONCLUSION

Validation overfitting represents a critical, yet often overlooked failure mode in modern deep learning pipelines, especially in ensemble learning settings where model selection and aggregation rely heavily on validation metrics. This article introduced GeNeX, a framework designed to explicitly address

VO by decoupling both model generation and ensemble construction from validation dependence. GeNeX operates in two distinct phases: the *GenE* module creates a diverse pool of candidate networks through a hybrid supervised-genetic strategy that bypasses validation monitoring entirely; the *ProtoNeX* module then clusters models based on predictive behavior, selects complementary experts within each cluster, and fuses their weights into robust cluster prototypes. By minimizing reliance on validation queries and enforcing both representational and behavioral diversity, GeNeX achieves strong generalization while systematically reducing VO gaps, even under extreme distribution shifts and large ensemble scales. Theoretical analysis and VO-aware evaluation protocols empirically support its robustness against both model-induced and dataset-induced VO risk. Additionally, extensive experiments across four real-world classification tasks, Skin Cancer, DeepFake, Plant Disease, and Pneumonia, demonstrate that GeNeX consistently outperforms SoA ensemble models in VO-prone scenarios. In addition to its strong performance, GeNeX demonstrates notable robustness under distributional shift, making it suitable for real-world deployments where validation overfitting is an inherent and critical risk.

Limitations and Future Work: While GeNeX has been validated on challenging real-world case study tasks, the scope of this work remains limited to the image classification domain. However, GeNeX is architecture- and modality-agnostic because it operates on model logits and behavior-driven predictive signatures rather than raw inputs; thus, only the base learner changes, while the GenE search and ProtoNeX clustering/prototyping logic remains the same. Future directions include extending the framework to other modalities such as segmentation and text classification. Concretely, for semantic segmentation, we would replace the base learner with a dense predictor such as U-Net, keep GenE unchanged as it breeds offspring models via weight fusion of task-trained bases, and apply ProtoNeX to per-pixel or pooled logits. For text classification, the base becomes a transformer classifier, GenE again recombines base weights to generate offspring, and ProtoNeX groups models by sequence-level logits. Finally, the proposed VO-aware evaluation protocol could serve as a standardized benchmark for assessing generalization robustness under realistic distributional-shifted environments, encouraging future research in this critical area.

REFERENCES

- [1] E. Pintelas, I. E. Livieris, V. Tampakas, and P. Pintelas, "MobileNet-HeX: Heterogeneous ensemble of MobileNet eXperts for efficient and scalable vision model optimization," *Big Data Cognit. Comput.*, vol. 9, no. 1, p. 2, Dec. 2024.
- [2] S. Lai et al., "Breaking long-tailed learning bottlenecks: A controllable paradigm with hypernetwork-generated diverse experts," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 7493–7520.
- [3] F. Xu, D. Chen, T. Jia, S. Deng, and H. Wang, "CBDMoE: Consistent-but-diverse mixture of experts for domain generalization," *IEEE Trans. Multimedia*, vol. 26, pp. 9814–9824, 2024.
- [4] U. Sarmah, P. Borah, and D. K. Bhattacharyya, "Ensemble learning methods: An empirical study," *Social Netw. Comput. Sci.*, vol. 5, no. 7, p. 924, Oct. 2024.
- [5] E. Pintelas, A. Koursaris, I. E. Livieris, and V. Tampakas, "TextNeX: Text network of eXperts for robust text classification—Case study on machine-generated-text detection," *Mathematics*, vol. 13, no. 10, p. 1555, May 2025.
- [6] C. Xu and S. Zhang, "A genetic algorithm-based sequential instance selection framework for ensemble learning," *Expert Syst. Appl.*, vol. 236, Feb. 2024, Art. no. 121269.
- [7] M. Kaya and Y. Çetin-Kaya, "A novel ensemble learning framework based on a genetic algorithm for the classification of pneumonia," *Eng. Appl. Artif. Intell.*, vol. 133, Jul. 2024, Art. no. 108494.
- [8] M. F. Dar and A. Ganiyada, "Deep learning and genetic algorithm-based ensemble model for feature selection and classification of breast ultrasound images," *Image Vis. Comput.*, vol. 146, Jun. 2024, Art. no. 105018.
- [9] E. Englesson and H. Azizpour, "Generalized Jensen–Shannon divergence loss for learning with noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 30284–30297.
- [10] J.-G. Gaudreault and P. Branco, "Empirical analysis of performance assessment for imbalanced classification," *Mach. Learn.*, vol. 113, no. 8, pp. 5533–5575, Aug. 2024.
- [11] S. Aburass and M. A. Rumman, "Quantifying overfitting: Introducing the overfitting index," in *Proc. Int. Conf. Electr., Comput. Energy Technol. (ICECET)*, Jul. 2024, pp. 1–7.
- [12] S. Boucheron, G. Lugosi, and P. Massart, *Concentration Inequalities: A Nonasymptotic Theory of Independence*. London, U.K.: Oxford Univ. Press, Feb. 2013.
- [13] G. Huang, Y. Li, G. Pleiss, Z. Liu, J. E. Hopcroft, and K. Q. Weinberger, "Snapshot ensembles: Train 1, get m for free," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
- [14] P. Izmailov, D. A. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson, "Averaging weights leads to wider optima and better generalization," in *Proc. Conf. Uncertainty Artif. Intell. (UAI)*, 2018, pp. 876–885.
- [15] P. Virtanen et al., "Scipy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [16] Y. Zhang et al., "Gaussian mixture model clustering with incomplete data," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 17, no. 1s, pp. 1–14, Jan. 2021.
- [17] J. Y. Choi and B. Lee, "Combining of multiple deep networks via ensemble generalization loss, based on MRI images, for Alzheimer's disease classification," *IEEE Signal Process. Lett.*, vol. 27, pp. 206–210, 2020.
- [18] T. T. Nguyen, N. Ye, and P. Bartlett, "Greedy convex ensemble," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, Jul. 2020, pp. 3101–3107.
- [19] T. Garipov, P. Izmailov, D. A. Podoprikin, D. Vetrov, and A. G. Wilson, "Loss surfaces, mode connectivity, and fast ensembling of DNNs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 8789–8798.
- [20] M. Wortsman, G. Ilharco, S. Gadre, R. Roelofs, R. Gontijo-Lopes, and A. S. Morcos, "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in *Proc. 39th Int. Conf. Mach. Learn.*, Baltimore, MD, USA, 2022, pp. 23965–23998.
- [21] E. Patel and D. S. Kushwaha, "Clustering cloud workloads: K-means vs Gaussian mixture model," *Proc. Comput. Sci.*, vol. 171, pp. 158–167, Jan. 2020.
- [22] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2022.
- [23] J. Healy and L. McInnes, "Uniform manifold approximation and projection," *Nature Rev. Methods Primers*, vol. 4, no. 1, p. 82, 2024.
- [24] J. Rashid, S. M. Boulaaras, M. S. Saleem, M. Faheem, and M. U. Shahzad, "A novel transfer learning approach for skin cancer classification on ISIC 2024 3D total body photographs," *Int. J. Imag. Syst. Technol.*, vol. 35, no. 2, p. 70065, Mar. 2025.
- [25] M. Hameed, A. Zameer, and M. A. Z. Raja, "A comprehensive systematic review: Advancements in skin cancer classification and segmentation using the ISIC dataset," *Comput. Model. Eng. Sci.*, vol. 140, no. 3, pp. 2131–2164, 2024.
- [26] B. Dolhansky et al., "The DeepFake detection challenge (DFDC) dataset," 2020, *arXiv:2006.07397*.
- [27] J. Huang, X. Wang, B. Du, P. Du, and C. Xu, "DeepFake MNIST+: A DeepFake facial animation dataset," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops (ICCVW)*, Oct. 2021, pp. 1973–1982.
- [28] (2020). *Real and Fake Face Detection Dataset*. [Online]. Available: <https://tinyurl.com/yc564acs>
- [29] E. Pintelas, I. E. Livieris, and P. E. Pintelas, "Quantization-based 3D-CNNs through circular gradual unfreezing for DeepFake detection," *IEEE Trans. Artif. Intell.*, vol. 6, no. 12, pp. 3351–3363, Dec. 2025.
- [30] A. Dobrovsky. (2022). *Plant Disease Classification Merged Dataset*. [Online]. Available: <https://tinyurl.com/yc89tcj3>
- [31] D. S. Kermany et al., "Identifying medical diagnoses and treatable diseases by image-based deep learning," *Cell*, vol. 172, no. 5, pp. 1122–1131, Feb. 2018.
- [32] Y. Xu, B. Du, and L. Zhang, "Robust self-ensembling network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3780–3793, Mar. 2024.
- [33] X. Gu et al., "Hierarchical weight averaging for deep neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 12276–12287, Sep. 2024.
- [34] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Learning from noisy labels with deep neural networks: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8135–8153, Nov. 2023.
- [35] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 7499–7519, Jun. 2024.
- [36] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.
- [37] T. Zhao, X. Guo, Y. Lin, and B. Du, "MixIR: Mixing input and representations for contrastive learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 5, pp. 8255–8264, May 2025.
- [38] G. Naidu, T. Zuva, and E. M. Sibanda, "A review of evaluation metrics in machine learning algorithms," in *Proc. Comput. Sci. On-line Conf.*, 2023, pp. 15–25.
- [39] S. García, A. Fernández, J. Luengo, and F. Herrera, "Advanced non-parametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power," *Inf. Sci.*, vol. 180, no. 10, pp. 2044–2064, May 2010.



Emmanuel Pintelas received the Ph.D. degree in mathematics and computer science from the Department of Mathematics, University of Patras, Patras, Greece, in 2023.

He is an Adjunct Lecturer at the Department of Statistics and Insurance Science, University of Piraeus, Greece. He has co-authored more than 20 publications in international scientific journals and seven conferences, while he was involved for internship scholarship and collaboration in the Archimedes Unit. His research interests include neural networks, machine learning, as well as time-series and knowledge extraction.



Ioannis E. Livieris received the Ph.D. degree in mathematics and computational intelligence from the University of Patras, Patras, Greece, in 2012.

He is an Associate Professor at the Department of Business and Organization Administration, University of Peloponnese, Greece. He has co-authored more than 65 publications in international scientific journals, 15 chapters in books, and 20 conferences and is/was involved in several dozens of national and European research projects. His research interests include neural networks, machine learning, as well as time-series and knowledge extraction.